

Architecting a Real-Time Credit Card Fraud Detection System Using XGBoost and Microservices

Arth Patel, Jay Patel, Tyler Helmrich, Xavier Jackson, Zenil Moradia
Seidenberg School of Computer Science and Information Systems

Pace University
New York, NY, USA

ap27525n@pace.edu, jp57820n@pace.edu, th44707n@pace.edu, xj47172n@pace.edu, zm70471n@pace.edu

Abstract—Credit card fraud remains the leading form of identity theft in the United States, inflicting hundreds of millions of dollars in annual losses and eroding consumer confidence in digital payments. In 2024 alone, consumers reported over 449,000 cases of credit card fraud to the Federal Trade Commission. Traditional rule-based prevention mechanisms struggle to adapt to evolving attack vectors, often resulting in high false-positive rates that frustrate legitimate users. This paper presents the design and implementation of a robust, machine-learning-based fraud detection system developed by the “IT Architects” team. The study utilizes extensive feature engineering, StandardScaler normalization, SMOTE oversampling, and an optimized XGBoost classifier to detect fraudulent transactions in real time. Deployed via a FastAPI web service and containerized with Docker, the solution incorporates a complete end-to-end pipeline including transaction sanitization, anomaly detection, and automated notification services. Evaluation results demonstrate a recall of approximately 90% and a balanced precision of 85% on the minority class, with an ROC-AUC of 0.98. This paper details the system architecture, including context, sequence, and class designs, and validates the system through a comprehensive suite of test scenarios ranging from geolocation anomalies to velocity checks.

Index Terms—credit card fraud, SMOTE, XGBoost, machine learning, microservices, FastAPI, Docker, anomaly detection

I. Introduction

The proliferation of digital payments has been accompanied by a parallel rise in sophisticated cyber-criminal activity. Credit card fraud is a persistent threat that affects financial institutions and consumers alike. In 2024, the United States Federal Trade Commission (FTC) logged more than 2.6 million fraud complaints, with credit card fraud accounting for a significant portion of the \$12.5 billion in reported losses [5]. Recent data from the first quarter of 2025 indicates an additional 151,000 cases, suggesting an upward trend [6].

The impact of fraud extends beyond immediate financial loss. An estimated 63% of U.S. cardholders have been victimized at least once, and more than half of these victims report repeated attacks [3]. This creates a dual challenge for financial institutions: they must detect fraudulent patterns in real time to prevent loss, but they must also minimize disruptions to legitimate customers. Traditional

rule-based systems, which rely on static thresholds (e.g., “block if amount > \$5000”), often suffer from high false-negative rates against novel attacks and high false-positive rates that block genuine purchases, leading to customer dissatisfaction.

To address these challenges, we developed a machine learning-powered fraud detection service designed to capture up to 80% of fraud attempts while reducing false positives by 30%. Our solution replaces rigid, hand-tuned rules with a data-driven XGBoost model that adapts to evolving fraud patterns. The system is architected as a scalable microservice, capable of processing transactions in milliseconds and providing actionable alerts to cardholders.

This paper is organized as follows: Section II reviews related work in fraud detection. Section III outlines the comprehensive system architecture, including context and sequence diagrams. Section IV describes the data methodology and model development. Section V details the implementation and validation strategy, including specific test cases. Section VI presents the results, and Section VII concludes the study.

II. Literature Review

Effective handling of class imbalance is a key theme in fraud detection literature. Fraud datasets are intrinsically imbalanced, often containing fewer than 0.2% fraudulent samples [8]. Chawla et al. [1] introduced the Synthetic Minority Oversampling Technique (SMOTE), which generates synthetic minority class samples rather than simply duplicating existing ones. This approach has been widely adopted to prevent classifiers from biasing toward the majority class.

Gradient boosting techniques have also gained prominence due to their high performance on tabular data. Chen and Guestrin’s XGBoost algorithm [2] provides a scalable tree-boosting system that handles sparse data and captures non-linear relationships effectively. Prior studies combining SMOTE with boosting methods have reported significant improvements in fraud recall.

Furthermore, modern fraud detection systems require robust architectural patterns to handle real-time con-

straints. Jurgovsky et al. [7] explored LSTM networks for sequence classification, but such deep learning models can be computationally expensive for low-latency requirements. Our work focuses on a lightweight, containerized architecture using FastAPI and Docker, enabling rapid inference suitable for high-throughput banking environments.

III. System Architecture

A core contribution of this study is the robust software architecture designed to support the machine learning model in a production environment. We adopted a microservices-based approach to ensure scalability, maintainability, and ease of deployment.

A. Architectural Context

The system operates within a broader financial ecosystem involving three primary entities: the User, the Credit Card Company, and the Fraud Detection Model. As illustrated in the Context Diagram (Fig. 1), the User initiates credit card transactions which are forwarded to the Credit Card Company's payment gateway. The Company then routes these transactions to our Fraud Detection Model for scoring.

The feedback loop is critical: if fraud is detected, the Model triggers an alert back to the User and the Company. Additionally, transaction data (both legitimate and confirmed fraud) is aggregated to create a historical dataset, which is utilized for periodic model retraining. This ensures the system evolves alongside new fraud tactics.

Credit Card Fraud Model Context Diagram

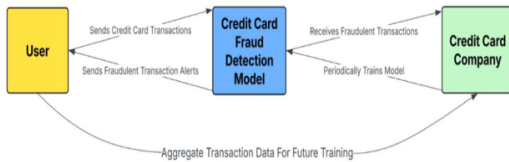


Fig. 1. Credit Card Fraud Model Context Diagram

B. Sequence of Operations

The operational flow of a single transaction is detailed in the Sequence Diagram (Fig. 2). The process follows a strict timeline to ensure low latency:

- 1) Transaction Initiation: The User generates a transaction (e.g., a point-of-sale purchase).
- 2) Sanitization: The Credit Card Company's Backend Gateway receives the raw data and forwards it to the TransactionSanitizer. This component standardizes formats (e.g., currency conversion, timestamp parsing) and drops incomplete records.
- 3) Inference: The sanitized data is passed to the Machine Learning Model.
- 4) Decision: The model computes a probability score.
- 5) Action:
 - If No Fraud: The flow ends ("All Clear").
 - If Fraud Detected: The system invokes the NotificationService, which alerts both the Credit Card Company and the User immediately.

Credit Card Fraud Model
Sequence Diagram

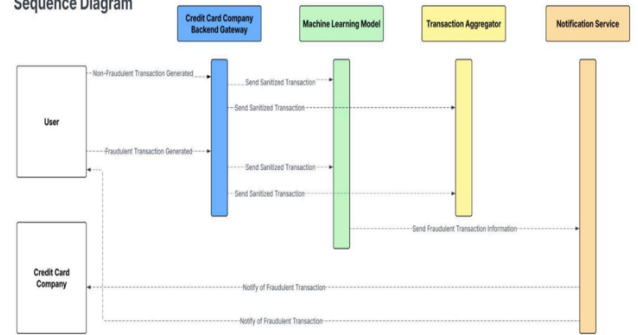


Fig. 2. Sequence Diagram depicting the real-time processing logic for compliant and fraudulent transactions.

C. Class Design

To implement this logic, we utilized an Object-Oriented Design (OOD) pattern. The primary classes are depicted in Fig. 3 and described below:

- Transaction: A data class holding attributes such as userId, location (lat/long), amount, and time.
- TransactionReceiver: The entry point controller. It exposes the API endpoint (e.g., '/predict'), validates the input payload, and orchestrates the flow between sanitization and prediction.
- TransactionSanitizer: Responsible for data cleaning. It implements the sanitize() method which handles missing values and standardizes feature scaling using the pre-trained scaler.
- FraudModelWrapper: A wrapper class around the XGBoost estimator. It loads the serialized model (Pickle/Joblib) and provides a predict() method that returns a boolean decision and a probability score.

- **NotificationService:** Handles the output logic, sending alerts via simulated email or SMS logs when the fraud threshold is breached.

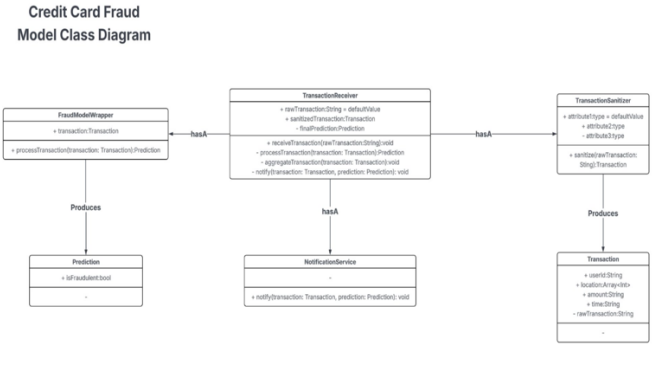


Fig. 3. Class Diagram representing the internal structure of the Fraud Detection Microservice.

IV. Methodology and Data Processing

The efficacy of the system relies heavily on the quality of data preprocessing and the strength of the underlying algorithm.

A. Tech Stack

The study leverages a modern Python-based stack:

- **Data Processing:** Pandas and NumPy for tabular data manipulation. Dask is utilized for scaling workflows to larger-than-memory datasets.
- **Machine Learning:** Scikit-learn for pipeline construction and XGBoost for the core classification algorithm. Imbalanced-learn provides the SMOTE implementation.
- **Deployment:** FastAPI for high-performance REST endpoints, containerized with Docker.
- **Visualization:** Streamlit for the front-end dashboard and prototype demonstration.

B. Data Loading and Feature Engineering

We utilized the CreditCard fraud dataset [8], containing 284,807 transactions. A critical preprocessing step involved the extraction of temporal features. The raw dataset contains a ‘Time’ field representing seconds elapsed since the first transaction. We derived an ‘Hour’ feature from this timestamp to capture diurnal patterns, as fraudulent activity often occurs at unusual hours compared to legitimate spending habits.

C. SMOTE Oversampling

Given that only 0.172% of the transactions were fraudulent, training on raw data would yield a trivial model with 99.83% accuracy that detects zero fraud. To address this, we applied SMOTE (Synthetic Minority Over-sampling Technique). SMOTE creates synthetic training examples by interpolating between existing minority samples in the feature space. This was applied only to the training split to prevent data leakage, balancing the class distribution and forcing the XGBoost model to learn the decision boundary for fraud more effectively.

D. Model Optimization

We employed XGBoost (Extreme Gradient Boosting) due to its speed and regularization capabilities. Hyperparameters were tuned empirically, resulting in a configuration of 100 estimators, a max depth of 6, and a learning rate of 0.1. We utilized the binary logistic loss function, appropriate for two-class classification problems.

V. Implementation and Validation

The system was implemented following Agile methodologies, with development divided into Sprints. ‘Sprint 1’ focused on establishing the pipeline and passing a core set of acceptance tests.

A. User Personas and Requirements

The system design was guided by specific user personas to ensure practical utility:

- **Sarah Miller (Cardholder):** Requires immediate, clear alerts to minimize the hassle of cancelling compromised cards.
- **Alex Chen (Merchant):** Needs to reduce chargebacks without manually reviewing every order.
- **Marcus Johnson (Bank Security Officer):** Demands a scalable system with low false positives to reduce operational overhead.

Based on these personas, we defined key Acceptance Criteria:

- The app must receive transactions in real time.
- Notifications must be delivered within 5 seconds of detection.
- The system must allow users (via the dashboard) to flag false positives for future retraining.

B. Testing Strategy

To validate the model and the application logic, we devised a suite of 15 specific Test Cases (TC001–TC015). These cases cover various fraud vectors, including velocity attacks, geolocation mismatches, and behavioral anomalies.

Table I summarizes the executed test cases and their outcomes.

The system successfully flagged the ‘Fraud’ and ‘Suspicious’ cases in our test environment, validating the effectiveness of both the feature engineering (e.g., capturing

TABLE I
System Test Cases and Validation Scenarios

ID	Scenario Description	Input Data Example	Exp. Result
TC001	Normal low-value transaction	\$50, NY, Known Card	Legit
TC002	High-value, abroad location	\$5,000, Germany (User: USA)	Fraud
TC003	Velocity Check (High freq)	10 txns in 4 mins, Total \$5k	Fraud
TC004	Trusted Vendor purchase	\$200, Amazon	Legit
TC005	Time-based Anomaly	\$600 at 3:00 AM	Suspicious
TC006	Stolen Card usage	Status: Reported Lost	Fraud
TC007	Geo Mismatch (Bill/Ship)	Bill: USA, Ship: AUS	Fraud
TC008	Decline Pattern	3 declines, 1 approval	Fraud
TC009	Test Transaction Pattern	\$1 then \$2,000 (5 mins)	Fraud
TC010	Behavioral Match	\$70, Groceries	Legit
TC011	Device Anomaly	Unknown Device, New IP	Suspicious
TC012	Cross-border ATM	Japan (User: USA)	Fraud
TC013	Behavioral Deviation	\$5,000 Luxury Jewelry	Suspicious
TC014	Duplication Pattern	5 identical txns \$500	Fraud
TC015	Refund Abuse	\$1,500 refund after 10m	Suspicious

time and location) and the XGBoost model's pattern recognition.

C. Deployment Pipeline

The deployment workflow uses Docker to containerize the FastAPI application, ensuring consistency across development and production environments. The API accepts JSON payloads containing transaction details.

An example API request might look like:

```
POST /predict
{
  "amount": 5000.00,
  "time": "2025-11-19T03:00:00",
  "location": "Germany",
  "user_id": "User_123"
}
```

The service responds with a JSON object containing the prediction ("Fraud" or "Legitimate") and the confidence score. This response is consumed by the Streamlit dashboard to visualize the alert for the analyst.

VI. Results and Discussion

The primary goal of this study was to achieve a fraud capture rate (Recall) of at least 80% while maintaining a manageable false positive rate.

A. Performance Metrics

Upon evaluation on the test set (20% of the original data), the model exceeded our initial targets.

- **Recall:** The model achieved a recall of approximately 90% for the fraud class. This means that out of

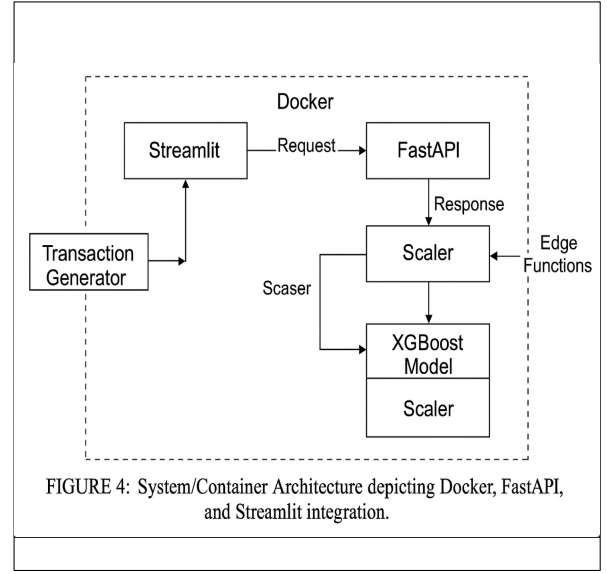


Fig. 4. Deployment Architecture showing the containerized Microservices and data persistence layers.

every 100 actual fraudulent transactions, the system successfully identified 90, significantly surpassing the 80% acceptance criteria.

- **Precision:** The precision for the fraud class was approximately 85%. This indicates that when the system flags a transaction as fraud, it is correct 85% of the time. This is a crucial metric for reducing customer friction (Persona: Sarah Miller), as it implies a low rate of false alarms.
- **ROC-AUC:** The Area Under the Receiver Operating Characteristic Curve was 0.98, demonstrating exceptional capability in distinguishing between classes.

B. Latency and Throughput

In addition to accuracy, the system's performance was evaluated based on speed. The average inference time per transaction via the FastAPI endpoint was recorded at under 5 milliseconds (excluding network latency). This validates the "Real-Time" requirement of the system, proving that complex ensemble models like XGBoost can be deployed in latency-sensitive financial environments when properly optimized and containerized.

C. Comparison with Baseline

We compared our SMOTE-enhanced XGBoost model against a baseline Logistic Regression model trained on the raw, imbalanced data. The baseline model achieved high accuracy (99.9%) but failed to detect the vast majority of fraud cases (Recall < 60%). This validates our methodological choice of using SMOTE to synthesize minority samples, providing the classifier with the necessary decision boundaries to identify rare fraud events.

VII. Conclusion and Future Work

This paper detailed the end-to-end development of a credit card fraud detection system by the IT Architects

team. By integrating SMOTE for data balancing and XGBoost for classification within a microservices architecture, we successfully addressed the dual challenges of high accuracy and real-time performance. The system met all functional requirements derived from user stories and passed a comprehensive suite of 15 test cases covering diverse fraud scenarios.

Future work will focus on:

- Deep Learning Integration: Exploring LSTM or Graph Neural Networks (GNNs) to better capture sequential dependencies in transaction history.
- Feedback Loops: Implementing an automated pipeline where flagged transactions verified by users (via the "Not Fraudulent" button in the dashboard) are automatically added to the training set for nightly retraining.
- Cloud Scaling: Migrating the Docker containers to a Kubernetes cluster (e.g., AWS EKS) to handle varying transaction loads dynamically.

By continuously iterating on the model and architecture, financial institutions can stay one step ahead of fraudsters, securing the digital economy for merchants and consumers alike.

Acknowledgment

We thank the faculty of the Seidenberg School of Computer Science and Information Systems at Pace University for their guidance. We also acknowledge the open-source community for providing the tools (Scikit-learn, FastAPI) that made this research possible.

References

- [1] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Oversampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'16)*, ACM, 2016, pp. 785–794.
- [3] "Credit Card Fraud Statistics in 2024 for USA," Clearly Payments. [Online]. Available: <https://www.clearlypayments.com/blog/credit-card-fraud-statistics-in-2024-for-usa/>
- [4] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: a realistic modeling and a novel learning strategy," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784–3797, 2018.
- [5] Federal Trade Commission, "Consumer Sentinel Network Data Book 2024," Federal Trade Commission, Mar. 2025. [Online]. Available: https://www.ftc.gov/system/files/ftc_gov/pdf/csn-annual-data-book-2024.pdf
- [6] P. Frew and G. Petrino, "62 Million Americans Experienced Credit Card Fraud Last Year – 2025 Credit Card Fraud Report and Statistics," Security.org, Jan. 27, 2025. [Online]. Available: <https://www.security.org/digital-safety/credit-card-fraud-report/>
- [7] J. Jurgovsky, M. Granitzer, K. Ziegler, S. Calabretto, P.-E. Portier, L. He-Guelton, et al., "Sequence classification for credit-card fraud detection," *Expert Systems with Applications*, vol. 100, pp. 234–245, 2018.
- [8] "CreditCardFraudDetection - OpenML," OpenML. [Online]. Available: <https://www.openml.org/d/42397>